

Stage 1 Lab - Human vs AI: Building Your First SmartCare Prototype

Learning objectives

- Create and run a simple Python file with basic input,output and processing statements
- Use lists, dictionaries and functions to enhance the Python file
- Build a small SmartCare appointment prototype.
- Use AI as a tutor rather than a replacement.
- Compare human-written and AI-generated code.
- Verify AI-generated code through execution and test inputs.
- Document a short AI-use reflection.

Files to create and commit in GitHub

```
stage01/  
  smartcare_v01.py  
  comparison.md  
  reflection.md  
  ai_usage.md
```

Part A - Understand the Problem: AI OFF

SmartCare needs a small prototype that allows a receptionist to record patient appointments. Each appointment records patient name, practitioner name and appointment time.

What data must be stored?

What functions might be useful?

What could go wrong?

What requirements are unclear?

Part B - Build a Human-Written Prototype: AI OFF

#task 1

Create and run a simple Python file with basic input,output statements

```
print("Welcome to SmartCare: Community Clinic Appointment Booking System!")
```

First Appointment

```
patient1_name = 'Alice Smith'
```

```
practitioner1_name = 'Dr. John Doe'
```

```
appointment1_time = '2024-07-20 10:00 AM'
```

```

print(f"Patient: {patient1_name} | Practitioner: {practitioner1_name} | Time:
{appointment1_time}")

# Second Appointment
patient2_name = 'Bob Johnson'
practitioner2_name = 'Dr. Jane Roe'
appointment2_time = '2024-07-20 11:30 AM'
print(f"Patient: {patient2_name} | Practitioner: {practitioner2_name} | Time:
{appointment2_time}")

```

#task1enhanced

Use lists, dictionaries and functions to enhance the Python file

```

appointments = []

def book_appointment(patient_name, practitioner_name, appointment_time):
    if not patient_name:
        raise ValueError("Patient name cannot be empty")
    appointment = {
        "patient": patient_name,
        "practitioner": practitioner_name,
        "time": appointment_time
    }
    appointments.append(appointment)

def display_appointments():
    if not appointments:
        print("No appointments recorded.")
        return
    for appointment in appointments:
        print(f"Patient: {appointment['patient']} | Practitioner:
{appointment['practitioner']} | Time: {appointment['time']}")

print("Welcome to SmartCare: The Clinical Appointment Booking System!")
book_appointment('Alice Smith', 'Dr. John Doe', '2024-07-20 10:00 AM')
book_appointment('Bob Johnson', 'Dr. Jane Roe', '2024-07-20 11:30 AM')
display_appointments()

```

^ Now, run both programs , and identify at least five limitations.

1. Practitioner name is not validated. The program checks whether the patient name is empty but it does not check whether the practitioner's name is empty
2. Appointment time is not validated. The program allows appointment_time to be empty or None because no validation is performed on it.
3. Double-booking is possible. The program does not check existing appointments before adding another appointment.
4. Data disappears when the program closes. It is not saved when it has been closed

5. Appointment time is only a string. Appointment is stored as plain text so the program does not check whether it is a valid date or time

Part C - Use AI as Tutor: AI ON (Use only UC approved GenAI Tool such as Microsoft CoPilot)

Suggested prompt structure:

Act as a Python tutor.

I am learning introductory software technology.

Here is a small appointment-booking function.

1. Explain what the code does.
2. Identify three limitations.
3. Suggest improvements.
4. Do not rewrite the whole application.
5. Ask me two questions to test my understanding.

1. What the code does

Concise takeaway: It prints two hard-coded appointments, then uses lists, dictionaries, and functions to store and display appointments dynamically.

Expanded explanation:

- The first section prints a welcome message and displays two appointments using fixed variables (patient1_name, etc.).
- The enhanced section introduces:
 - A list called appointments to store multiple bookings.
 - A function book_appointment that:
 - Validates the patient name.
 - Creates a dictionary with patient, practitioner, and time.
 - Appends it to the appointments list.
 - A function display_appointments that:
 - Prints all stored appointments, or a message if none exist.
- Finally, it books two appointments and displays them.

This shows the transition from static output to reusable, data-driven logic using lists, dictionaries, and functions.

2. Three limitations

- **Hard-coded data** — All appointments are manually typed in; there is no user input or dynamic interaction.
- **No conflict checking** — The system does not prevent double-booking the same practitioner at the same time.
- **No persistence** — Appointments disappear when the program ends because nothing is saved to a file or database.

3. Suggested improvements

- **Add user input** Let users type patient names, practitioners, and times instead of hard-coding them.
- **Implement validation** Check for missing fields, invalid dates, or scheduling conflicts.
- **Add storage** Save appointments to a file (CSV/JSON) so they persist between runs.

- **Improve structure** Consider using a class like Appointment or ClinicSystem for cleaner organization as the program grows.

(These are conceptual improvements — not a rewrite.)

4. Two questions to test your understanding

1. If you removed the `appointments.append(appointment)` line, what would happen when you call `display_appointments`
2. Why is a dictionary a good data structure for storing each appointment's details

Part D - Generate an Alternative: AI ON

Ask AI to create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. Explicitly prohibit a database or GUI.

```
appointments = []

def add_appointment(patient_name, practitioner_name, appointment_time):

    """Store a basic appointment using simple Python data structures."""

    appointment = {

        "patient": patient_name,

        "practitioner": practitioner_name,

        "time": appointment_time }

    appointments.append(appointment)

    print("Appointment added successfully!")
```

Part E - Compare Human and AI Versions

Question	Human version	AI version
Easy to understand?	Yes. Straightforward	Yes it is short
Runs successfully?	Yes for normal appointments	Yes for normal values
Uses only required features?	Yes	Yes. Does not add a database or GUI
Adds assumptions?	It assumes storing names and time is sufficient	Does not manage a list of appointments
Handles errors?	Only checks patient name	Also only checks the patient name
Could I explain it?	Yes	Kind of

Part F - Verify Behaviour

- Normal appointment
- Blank patient name

- Two appointments for the same practitioner/time
- Strange input such as patient_name=None or appointment_time=None

Part G - Improve One Thing

Choose exactly one controlled improvement, for example: if not patient_name: raise ValueError("Patient name cannot be empty")

Part H - Reflection (150-250 words)

What did you build before using AI?

What did AI help you understand?

Did AI make assumptions?

How did you verify the AI output?

What engineering work remained for you?

Before AI I worked with the smartcare appointment prototype and explored how patient names, practitioner names and appointment times could be stored and displayed. I also ran the enhanced version with the list, dictionaries and functions and identified 5 limitations including possible double booking.

I then used Microsoft Copilot as a tutor to explain the code, show possible mistakes and suggest improvements. Some of its suggestions were useful like the suggestion about validating the input. However AI could also make assumptions about features that were not correct so I could not automatically accept every suggestion.

I verified the code by running it with normal and unusual values including an empty patient name. None values and two appointments with the same practitioner and time. Testing showed that appointment_time=None was accepted so I added validation for the appointment time and tested it again. Had to think how appointments should be validated or displayed.

Submission checklist [GitHub Commit]

- Python file runs.
- Comparison table completed.
- Normal and unusual inputs tested.
- AI assistance documented.
- Reflection completed.
- I can explain my code.